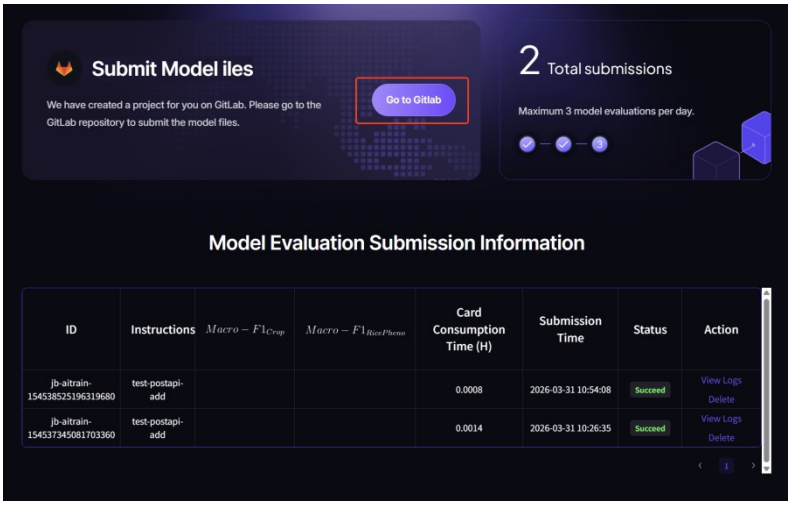
Round 2 Project Submission Guide

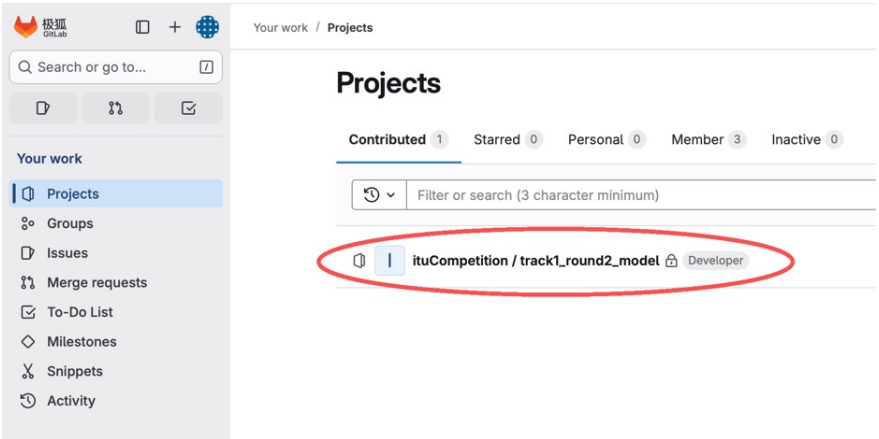
1. Enter the GitLab Repository

Go to the official competition website and click the "submission" button on the top navigation bar to enter the submission page. The first time you enter this page, you need to select a track; once selected, it cannot be changed, so choose carefully.

After selecting the track, click "Go to GitLab" to enter the repository.



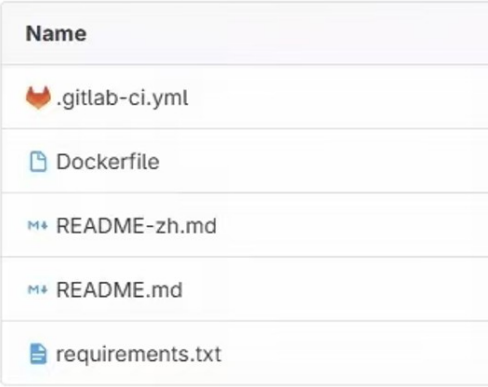
Depending on the track, the repository is named "track1_round2_model", "track2_round2_model", and "track3_round2_model". Please confirm that the repository matches the track you selected.



Note:

- Only pushes to the "main" branch will trigger an automatic submission.
- There are only 3 submission attempts per day.
- The platform is inference-only, not a training environment.
- The output must be written to /output/result.json.

2. File Setup



Only templates for .gitlab-ci.yml, Dockerfile, and requirements are provided. You need to implement data loading, feature extraction, model inference, and result output logic yourself.

Note:

The input data structure is:

Sample_track1

/input/test_point.csv

/input/region_test/*.tiff

2.1. .gitlab-ci.yml Configuration

.gitlab-ci.yml is the pipeline configuration file. You only need to modify the path mounted into the image and the startup command.

Where:

- Set the path field under DATASETS to /input.
- Set the outputPath field under OUTPUT_DIR to /output.
- Set the value of OUTPUT_DIR under ENV_VARS to /output.

- Update START_CMD according to your actual command (recommended to use ./run.sh as a unified entry point).

.gitlab-ci.yml 8.24 KiB

```yaml
# GitLab CI/CD 流水线配置
stages:
  - build
  - deploy

variables:
  DOCKER_IMAGE: "10.200.99.202:15080/zero2x002/zero2x_repository:${CI_COMMIT_SHORT_SHA}-${CI_PIPELINE_ID}"
  KUBECONFIG: "/tmp/kubeconfig"
  DOCKER_CACHE_IMAGE: "10.200.99.202:15080/zero2x002/zero2x_repository:cache-${CI_PROJECT_ID}"

  # "path":"/mnt/si000886fq1w/default/dataset/semifinalDataset" 此为南湖挂载的数据集路径，可自行修改，代码中的数据集路径需修改为此路径
  DATASETS: '[{"name":"itu赛道3训练数据集","uuid":"113bf8b6-13f6-48ce-8010-4310d22db7fa","path":"/input","version":"V1"}]'
  OUTPUT_DIR: '{"uuid":"f57d771e-ea0c-40a0-a161-d7b150695df6","bucket":"default","path":"output/track3/$GITLAB_USER_EMAIL","outputPath":"/output",
  ENV_VARS: '[{"key":"PYTHONUNBUFFERED","value":"1"},{"key":"OUTPUT_DIR","value":"/output"}]'

  # 启动命令
  START_CMD: "./run.sh"

  # 算法参数
  ALGO_NAME: "赛道3测试算法"
  ALGO_TYPE: "train"
  SRC_TYPE: "image"
  ALGO_DESCRIPTION: "赛道3测试算法"
```

2.2. Dockerfile Configuration

The default Dockerfile pulls the competition-base base image. You should debug and run your code based on this image.

Dockerfile 954 B

```dockerfile
# 10.200.99.202:15080/zero2x002/competition-base:ubuntu22.04-cuda12.3.2-cudnn9-py310.19
FROM 10.200.99.202:15080/zero2x002/competition-base:ubuntu22.04-py310.19
WORKDIR /app

# 接收构建参数，指定 pip 缓存目录（默认值可留空）
ARG PIP_CACHE_DIR

# 将依赖文件单独复制（利用 Docker 层缓存）
COPY requirements.txt .

# 安装依赖，使用传入的缓存目录（如果 PIP_CACHE_DIR 有值，则使用 --cache-dir）
# 注意：如果 PIP_CACHE_DIR 为空，则仍会使用系统缓存（但无外部缓存时效果有限）
RUN if [ -n "$PIP_CACHE_DIR" ]; then \


        pip install --cache-dir $PIP_CACHE_DIR -r requirements.txt -i https://repo.huaweicloud.com/repository/pypi/simple; \
    else \
        pip install -r requirements.txt -i https://repo.huaweicloud.com/repository/pypi/simple; \
    fi

COPY . .

ENV FLASK_APP=train.py
CMD ["flask", "run", "--host=0.0.0.0", "--port=30194"]
```

According to the official competition documentation, WORKDIR should be set to /workspace. The last two lines are for testing only and can be removed. This Dockerfile will copy all files in the repository into /workspace in the image. Be sure to grant execute permission to the files to be executed to avoid errors (for example, RUN chmod +x run.sh).

Based on the above, the Dockerfile (example) should be updated as follows:

Dockerfile 901 B

```dockerfile
# 10.200.99.202:15080/zero2x002/competition-base:ubuntu22.04-cuda12.3.2-cudnn9-py310.19
FROM 10.200.99.202:15080/zero2x002/competition-base:ubuntu22.04-py310.19
WORKDIR /workspace

# 接收构建参数，指定 pip 缓存目录（默认值可留空）
ARG PIP_CACHE_DIR

# 将依赖文件单独复制（利用 Docker 层缓存）
COPY requirements.txt .

# 安装依赖，使用传入的缓存目录（如果 PIP_CACHE_DIR 有值，则使用 --cache-dir）
# 注意：如果 PIP_CACHE_DIR 为空，则仍会使用系统缓存（但无外部缓存时效果有限）
RUN if [ -n "$PIP_CACHE_DIR" ]; then \


        pip install --cache-dir $PIP_CACHE_DIR -r requirements.txt -i https://repo.huaweicloud.com/repository/pypi/simple; \
    else \
        pip install -r requirements.txt -i https://repo.huaweicloud.com/repository/pypi/simple; \
    fi

COPY . .

RUN chmod +x run.sh
```

2.3. run.sh Configuration (Example; Not Included in the Initial Folder)

The contents of run.sh are as follows:

run.sh 82 B

```
python3 scan_files.py
python3 inversion.py --data_dir /input --result_dir /output

```

3. Code Submission Notes

This system uses a CI/CD automated deployment and testing process. The platform input is not a training-set structure but an inference-only structure. When code is pushed to the "main" branch, the deployment process is triggered automatically and one submission attempt is consumed. Each team can submit only three times per day, so please make sure your code and the automation files (.gitlab-ci.yml and Dockerfile) are configured correctly before submitting.

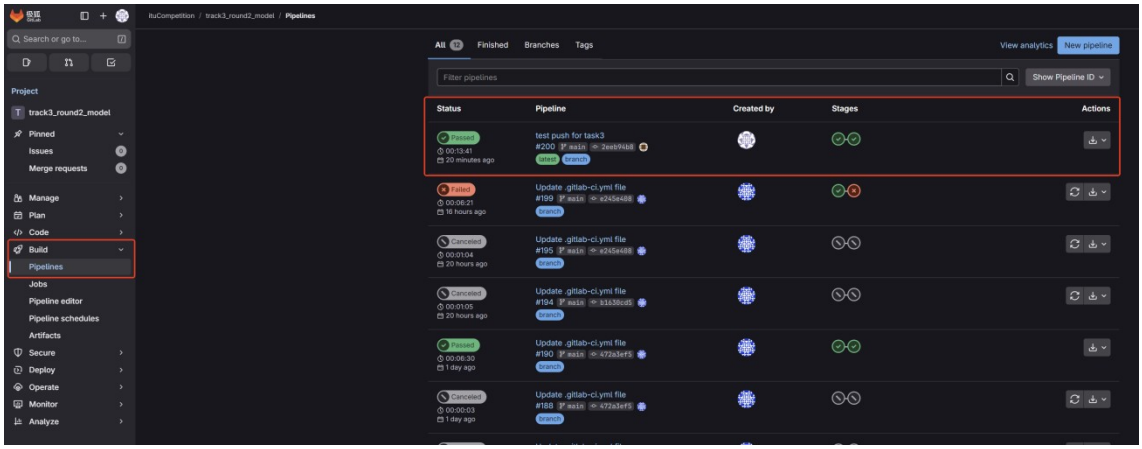
3.1. Submission Example

Name	Last commit	Last update
.gitlab-ci.yml	test push for task3	16 hours ago
Dockerfile	test push for task3	16 hours ago
README.md	Initial commit	1 day ago
inversion.py	test push for task3	16 hours ago
requirements.txt	Add new file	1 day ago
run.sh	test push for task3	16 hours ago
scan_files.py	test push for task3	16 hours ago

As shown above, compared with the initial repository, the following are added:

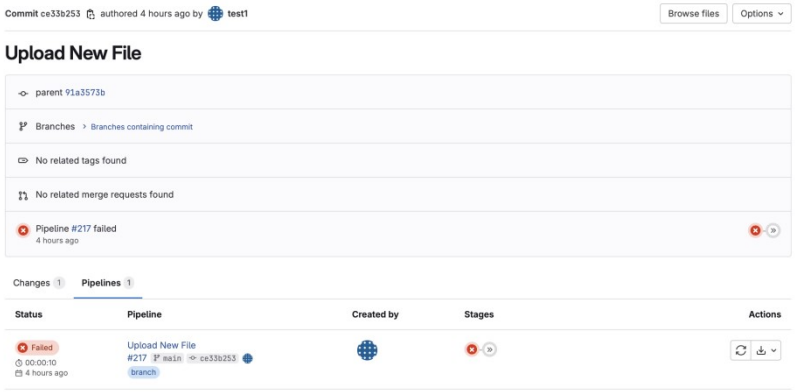
- scan_files.py: helper script to traverse the contents of /input.
- inversion.py: inference script that outputs result.json.
- run.sh: command entry point.

After pushing code to the "main" branch, you can see the automatic image build progress in the pipeline:



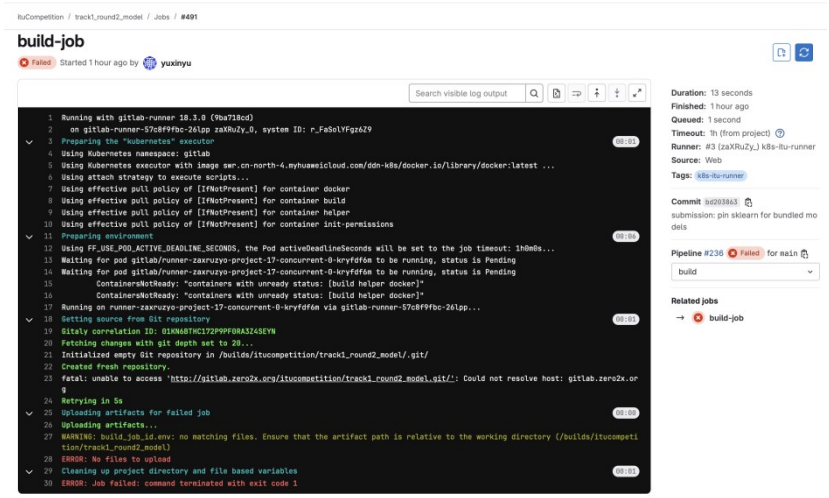
3.2. Examples of Submission Failures

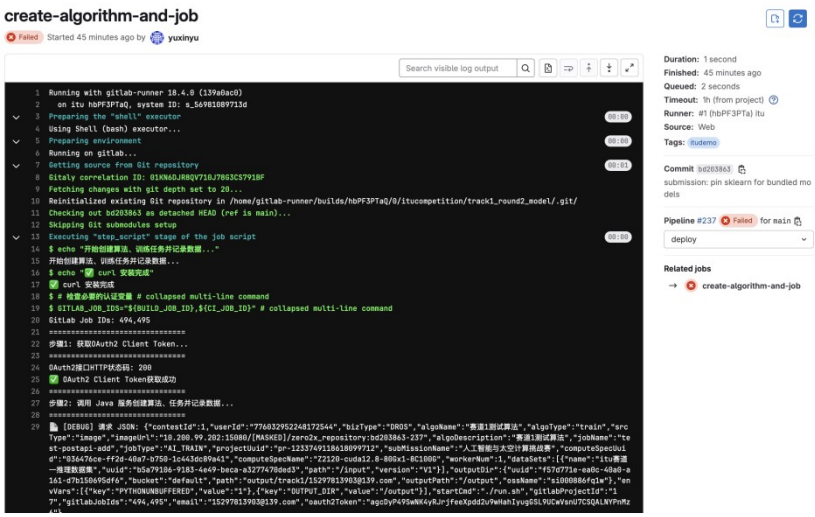
3.2.1. Upload via the Web UI



3.2.2. Build-job/create-algorithm-and-job failed

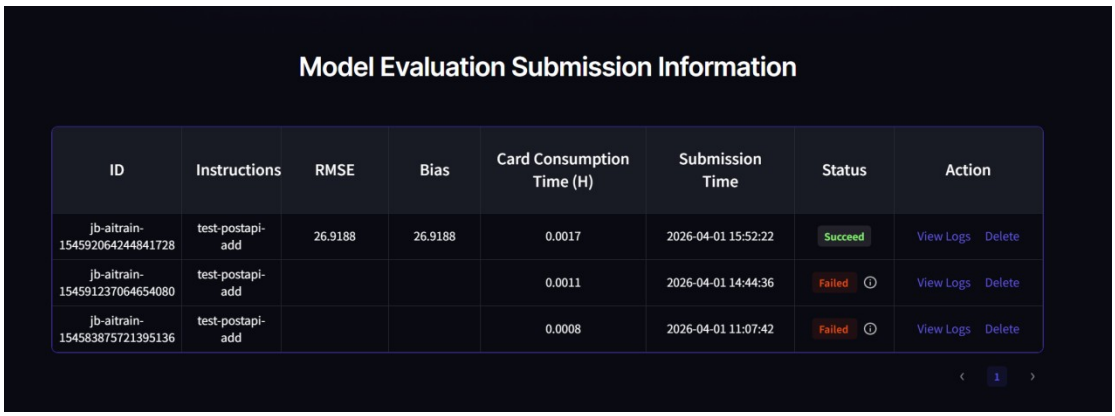
Usually it will resolve itself after waiting for a while.





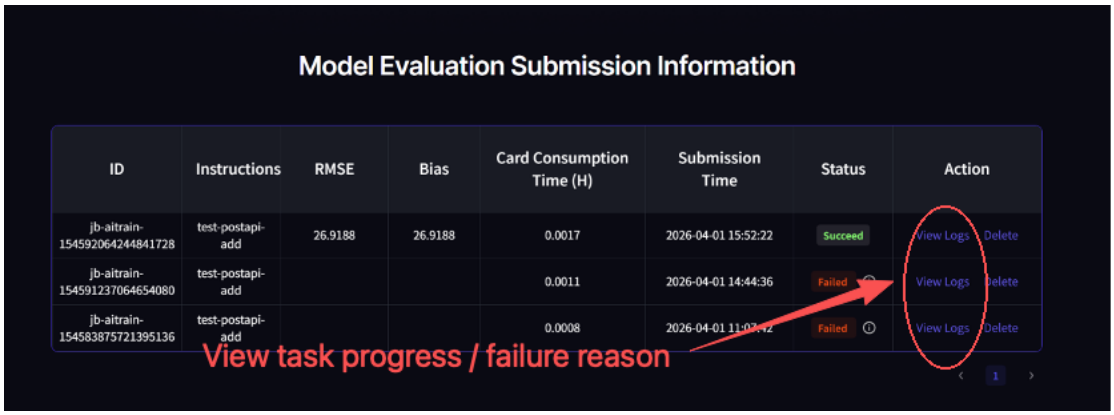
3.3. Execution Example

After the image is built, it will automatically enter the execution queue. Queue execution may take from a few minutes to tens of minutes. After it finishes, you can see the submission result on the competition website under "submission":



3.4. Examples of Execution Failures

Use the log to view task execution progress and the failure reason (scroll to the bottom of the page):



3.4.1. There is an issue with the data loading script,

```
2026-04-02 11:13:40.583108123 nm129-a100-80g-55 /input/region_test/region58_2018-07-26-00-00_2018-07-26-23-59_Sentinel-2_L2A_B02_(Raw).tiff
2026-04-02 11:13:40.583110693 nm129-a100-80g-55 /input/region_test/region14_2018-07-28-00-00_2018-07-28-23-59_Sentinel-2_L2A_B8A_(Raw).tiff
2026-04-02 11:13:40.583113196 nm129-a100-80g-55 /input/region_test/region19_2018-10-09-00-00_2018-10-09-23-59_Sentinel-2_L2A_B12_(Raw).tiff
2026-04-02 11:13:40.583115712 nm129-a100-80g-55 /input/region_test/region03_2018-06-21-00-00_2018-06-21-23-59_Sentinel-2_L2A_B04_(Raw).tiff
2026-04-02 11:13:40.583118213 nm129-a100-80g-55 /input/region_test/region19_2018-09-26-00-00_2018-09-26-23-59_Sentinel-2_L2A_B12_(Raw).tiff
2026-04-02 11:13:40.583120783 nm129-a100-80g-55 /input/region_test/region55_2018-11-13-00-00_2018-11-13-23-59_Sentinel-2_L2A_B8A_(Raw).tiff
2026-04-02 11:13:40.583123318 nm129-a100-80g-55 /input/region_test/region19_2018-09-26-00-00_2018-09-26-23-59_Sentinel-2_L2A_B05_(Raw)(1).tiff
2026-04-02 11:13:40.583125854 nm129-a100-80g-55 /input/region_test/region14_2018-07-23-00-00_2018-07-23-23-59_Sentinel-2_L2A_B02_(Raw).tiff
2026-04-02 11:13:40.583128404 nm129-a100-80g-55 /input/region_test/region30_2018-04-07-00-00_2018-04-07-23-59_Sentinel-2_L2A_B06_(Raw).tiff
2026-04-02 11:13:40.583130886 nm129-a100-80g-55
2026-04-02 11:13:40.583133664 nm129-a100-80g-55 遍历完成！
2026-04-02 11:13:40.583136977 nm129-a100-80g-55 ========== 开始运行推理脚本 ==========
2026-04-02 11:13:41.702392239 nm129-a100-80g-55 Traceback (most recent call last):
2026-04-02 11:13:41.702408217 nm129-a100-80g-55 File "/workspace/inference.py", line 137, in <module>
2026-04-02 11:13:41.702443351 nm129-a100-80g-55 main()
2026-04-02 11:13:41.702458300 nm129-a100-80g-55 File "/workspace/inference.py", line 127, in main
2026-04-02 11:13:41.702466949 nm129-a100-80g-55 pred_row_json = run_pipeline(data_dir=data_dir, result_dir=result_dir)
2026-04-02 11:13:41.702468759 nm129-a100-80g-55 File "/workspace/inference.py", line 26, in run_pipeline
2026-04-02 11:13:41.702476610 nm129-a100-80g-55 dataset_root = find_dataset_root(data_dir)
2026-04-02 11:13:41.702487819 nm129-a100-80g-55 File "/workspace/inference.py", line 20, in find_dataset_root
2026-04-02 11:13:41.702507233 nm129-a100-80g-55 raise FileNotFoundError(
2026-04-02 11:13:41.702509613 nm129-a100-80g-55 FileNotFoundError: Could not find dataset-4 under /input. Expected train/points_train_label.csv and test/points_test.csv.
```

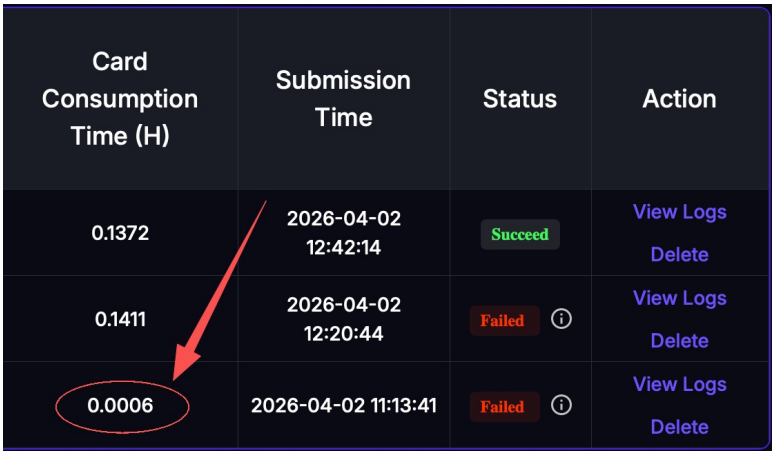
Cause of the error in this example:

/input was treated as a train + test directory, but what the platform actually m ounts (i.e., the input data structure) is:

/input/test_point.csv

/input/region_test/*.tiff

If a submission fails very early, it is most likely that the data loading has a problem:



Card Consumption Time (H)	Submission Time	Status	Action
0.1372	2026-04-02 12:42:14	Succeed	View Logs Delete
0.1411	2026-04-02 12:20:44	Failed	View Logs Delete
0.0006	2026-04-02 11:13:41	Failed	View Logs Delete

3.4.2. Dependency Version Issues

The dependencies to be installed must strictly match the versions actually used in the project.

```
2026-04-02 12:20:43.210793364 nm129-a100-80g-55 /input/region_test/region14_2018-07-23-00-00_2018-07-23-23-59_Sentinel-2_L2A_B02_(Raw).tiff
2026-04-02 12:20:43.210796336 nm129-a100-80g-55 /input/region_test/region30_2018-04-07-00-00_2018-04-07-23-59_Sentinel-2_L2A_B06_(Raw).tiff
2026-04-02 12:20:43.210801252 nm129-a100-80g-55
2026-04-02 12:20:43.210806734 nm129-a100-80g-55 遍历完成！
2026-04-02 12:20:43.210812078 nm129-a100-80g-55 ========== 开始运行推理脚本 ==========
2026-04-02 12:20:44.266873166 nm129-a100-80g-55 [input] detected round-2 platform layout: test csv=/input/test_point.csv, region dir=/input/region_test
2026-04-02 12:20:44.308993431 nm129-a100-80g-55 [meta] 0/2235
2026-04-02 12:20:46.124649284 nm129-a100-80g-55 [meta] 200/2235
2026-04-02 12:20:48.035867699 nm129-a100-80g-55 [meta] 400/2235
2026-04-02 12:20:49.809662888 nm129-a100-80g-55 [meta] 600/2235
2026-04-02 12:20:51.396598102 nm129-a100-80g-55 [meta] 800/2235
2026-04-02 12:20:53.074829272 nm129-a100-80g-55 [meta] 1000/2235
2026-04-02 12:20:54.666680008 nm129-a100-80g-55 [meta] 1200/2235
2026-04-02 12:20:56.316624936 nm129-a100-80g-55 [meta] 1400/2235
2026-04-02 12:20:58.044041703 nm129-a100-80g-55 [meta] 1600/2235
2026-04-02 12:20:59.707166167 nm129-a100-80g-55 [meta] 1800/2235
2026-04-02 12:21:01.399404767 nm129-a100-80g-55 [meta] 2000/2235
2026-04-02 12:21:03.093360016 nm129-a100-80g-55 [meta] 2200/2235
2026-04-02 12:21:03.429709895 nm129-a100-80g-55 [meta] 2235/2235
2026-04-02 12:21:03.492865209 nm129-a100-80g-55 [feature] platform_test 0/942
2026-04-02 12:25:19.059622730 nm129-a100-80g-55 [feature] platform_test 500/942
2026-04-02 12:29:10.156780403 nm129-a100-80g-55 [feature] platform_test 942/942
2026-04-02 12:29:10.200018501 nm129-a100-80g-55 /usr/local/lib/python3.10/site-packages/sklearn/base.py:442: InconsistentVersionWarning: Trying to unpickle estimator SimpleImputer from version 1.5.1 when using version 1.7.2. This might lead to breaking code or invalid results. Use at your own risk. For more info please refer to:
2026-04-02 12:29:10.200060281 nm129-a100-80g-55 https://scikit-learn.org/stable/model_persistence.html#security-maintainability-limitations
2026-04-02 12:29:10.200069113 nm129-a100-80g-55 warnings.warn(
2026-04-02 12:29:10.200076786 nm129-a100-80g-55 /usr/local/lib/python3.10/site-packages/sklearn/base.py:442: InconsistentVersionWarning: Trying to unpickle estimator Pipeline from version 1.5.1 when using version 1.7.2. This might lead to breaking code or invalid results. Use at your own risk. For more info please refer to:
2026-04-02 12:29:10.200083707 nm129-a100-80g-55 https://scikit-learn.org/stable/model_persistence.html#security-maintainability-limitations
2026-04-02 12:29:10.200090560 nm129-a100-80g-55 warnings.warn(
2026-04-02 12:29:10.201043187 nm129-a100-80g-55 Traceback (most recent call last):
2026-04-02 12:29:10.201061043 nm129-a100-80g-55 File "/workspace/inference.py", line 383, in <module>
2026-04-02 12:29:10.201143319 nm129-a100-80g-55 main()
2026-04-02 12:29:10.201150630 nm129-a100-80g-55 File "/workspace/inference.py", line 373, in main
2026-04-02 12:29:10.201209407 nm129-a100-80g-55 pred_row_json = run_pipeline(data_dir=data_dir, result_dir=result_dir)
2026-04-02 12:29:10.201219925 nm129-a100-80g-55 File "/workspace/inference.py", line 341, in run_pipeline
2026-04-02 12:29:10.201239903 nm129-a100-80g-55 return run_platform_pipeline(test_point_csv=test_point_csv, region_test_dir=region_test_dir, result_dir=result_dir)
2026-04-02 12:29:10.201247278 nm129-a100-80g-55 File "/workspace/inference.py", line 326, in run_platform_pipeline
2026-04-02 12:29:10.201291430 nm129-a100-80g-55 return predict_platform_rows(features_df=features_df, model_dir=model_dir, work_dir=work_dir)
2026-04-02 12:29:10.201299240 nm129-a100-80g-55 File "/workspace/inference.py", line 188, in predict_platform_rows
2026-04-02 12:29:10.201323925 nm129-a100-80g-55 crop_model = joblib.load(crop_model_path)
2026-04-02 12:29:10.201333672 nm129-a100-80g-55 File "/usr/local/lib/python3.10/site-packages/joblib/numpy_pickle.py", line 749, in load
2026-04-02 12:29:10.201428226 nm129-a100-80g-55 obj = _unpickler(
2026-04-02 12:29:10.201435534 nm129-a100-80g-55 File "/usr/local/lib/python3.10/site-packages/joblib/numpy_pickle.py", line 626, in _unpickle
2026-04-02 12:29:10.201504005 nm129-a100-80g-55 obj = unpickler.load()
2026-04-02 12:29:10.201511137 nm129-a100-80g-55 File "/usr/local/lib/python3.10/pickle.py", line 1213, in load
2026-04-02 12:29:10.201652611 nm129-a100-80g-55 dispatch[key[0]](self)
2026-04-02 12:29:10.201659683 nm129-a100-80g-55 File "/usr/local/lib/python3.10/pickle.py", line 1538, in load_stack_global
2026-04-02 12:29:10.201810434 nm129-a100-80g-55 self.append(self.find_class(module, name))
2026-04-02 12:29:10.201817227 nm129-a100-80g-55 File "/usr/local/lib/python3.10/pickle.py", line 1582, in find_class
2026-04-02 12:29:10.201908648 nm129-a100-80g-55 return _getattribute(sys.modules[module], name)[0]
2026-04-02 12:29:10.201975317 nm129-a100-80g-55 File "/usr/local/lib/python3.10/pickle.py", line 331, in _getattribute
2026-04-02 12:29:10.202024404 nm129-a100-80g-55 raise AttributeError("Can't get attribute {!r} on {!r}"
2026-04-02 12:29:10.202031616 nm129-a100-80g-55 AttributeError: Can't get attribute '_RemainderColsList' on <module 'sklearn.compose._column_transformer' from '/usr/local/lib/python3.10/site-packages/sklearn/compose/_column_transformer.py'>
```

3.4.3. Example Repository Contents (the repository submitted during testing; the actual submission repository structure may differ):

Name	Type/Location	Description
models/	Weights Directory	Stores built-in model weights. Currently these are the .joblib files required for Round 2 inference-only; they are loaded directly when the container runs and are not trained on the fly.
.gitlab-ci.yml	Pipeline Configuration File	GitLab submission pipeline configuration: pulls code, builds the image, pushes the image, and calls the platform API to create a zero2x task.
Dockerfile	Image Build File	Defines how the submission image is built. It mainly installs dependencies, copies code and models, and sets the working directory for the platform to start the container and run it.
README.md	Project Documentation	Used to describe the submission structure, how to run the project, or platform requirements.
classification.py	Core Script	Core baseline classification script, including two-stage model training/prediction logic: stage 1 predicts crop type, stage 2 predicts phenological stage; it also exports the .joblib model and per-row prediction results.
extract_features.py	Feature Extraction Script	Extracts features from the point table (/input/test_point.csv) and Sentinel-2 TIFFs (/input/region_test/*.tiff), maps latitude/longitude points to image pixels, and generates the tabular features required by the model.
extract_meta.py	Metadata Extraction Script	Scans TIFF images and extracts metadata, mainly organizing region, date, bands, geographic extent, and other information for subsequent feature extraction.
inference.py	Inference Entry Script	Official submission entry point. Detects the platform input structure, calls feature extraction, loads the weights under models/, and generates /output/result.json.
requirements.txt	Dependency List	List of Python dependencies. Must be compatible with the model weights.
result.json	Sample Output	Used to illustrate the final result format

	File	expected by the evaluation system; it is not part of the training code itself.
run.sh	Startup Script	The actual script executed after the container starts, usually prints/scans the input first and then calls inference.py.
scan_files.py	Debug Helper Script	Used to traverse /input and print the files actually mounted by the platform, making it easier to troubleshoot input-structure issues.

4. Competition GPU Base Runtime Environment Image (PyTorch Version) Guide

4.1. Key Image Information

project	Detailed information
Mirror Name	competition-base:pytorch2.5.1-cuda12.1-cudnn9
base image	docker.1ms.run/pytorch/pytorch:2.5.1-cuda12.1-cudnn9-devel (GPU image pre-installed with PyTorch)
System version	Ubuntu 22.04 LTS（Jammy Jellyfish）
Python version	Consistent with the official PyTorch mirror (compatible with Python version PyTorch 2.5.1)
CUDA version	12.1 (Supports NVIDIA Ampere/ADA/Hopper architecture GPUs)
cuDNN version	9.0 (GPU-accelerated deep learning inference/training)
Pre-installed core dependencies	PyTorch=2.5.1、torchvision、torchaudio、numpy=2.1.2、pandas=2.3.3、scipy=1.15.3、rasterio=1.4.4、pvlib=0.15.0、opencv-python-headless=4.13.0.92、openpyxl=3.1.2、scikit-learn=1.7.2
Pre-installed system tools	wget、curl、vim、unzip、ca-certificates
Core Directory	/input (input data directory), /output (output results directory), /workspace (code working directory)
Permissions Description	The `/input//output` directory has 777 permissions by default, allowing any user to read and write to it.
GPU support	Requires the host machine to have NVIDIA Driver 535+ (compatible with CUDA 12.1) and nvidia-docker2 installed.

4.2. Image Purpose

This image is the unified GPU base runtime environment for the competition (PyTorch pre-installed). It is suitable for:

- GPU-based PyTorch model training/inference (native support for PyTorch 2.5.1 with GPU acceleration; no manual installation required).

- Data format conversion (supports common formats such as Excel/.xlsx, JSON, CSV).
- Working with the evaluation image (RMSE/MAE calculation) to evaluate model output results.
- Compatible with CPU mode (automatically falls back to CPU when no GPU is available).

4.3. Prerequisites

4.3.1. Host GPU Environment Requirements (Important for organizers/participants)

4.3.2. Hardware Requirements

- NVIDIA GPU (compute capability ≥ 7.0, e.g., Tesla T4/V100/A10/A100, RTX 3090/4090).
- VRAM ≥ 8 GB (16 GB+ recommended).

4.3.3. Software Requirements (must be installed on the host)

```
# 1. Install NVIDIA GPU driver (version ≥ 535, compatible with CUDA 12.1)
nvidia-smi # Verify the driver; if GPU info is shown, the driver is working
# 2. Install nvidia-docker2
distribution=$(. /etc/os-release;echo $ID$VERSION_ID)
curl -s -L https://nvidia.github.io/nvidia-docker/gpgkey | sudo apt-key add -
curl -s -L https://nvidia.github.io/nvidia-docker/$distribution/nvidia-docker.list | sudo tee /etc/apt/sources.list.d/nvidia-docker.list
sudo apt-get update && sudo apt-get install -y nvidia-docker2
sudo systemctl restart docker
# 3. Verify nvidia-docker (requires the corresponding base image)
docker run --rm --gpus all docker.1ms.run/pytorch/pytorch:2.5.1-cuda12.1-cudnn9-devel nvidia-smi
```

4.3.4. Verify the GPU environment

If the above command outputs GPU model, driver version, CUDA version, etc., the host GPU environment is working properly.

4.3.5. Load the Image

Upload the image file (competition-base-pytorch.tar) to the server/local environment and run the following commands to load it:

```
# Decompress the image package (if not decompressed)
gzip -d competition-base-pytorch.tar.gz
# Load the local GPU image package
```

```
docker load -i competition-base-pytorch.tar
# Verify that the image is loaded successfully
docker images | grep competition-base
```

4.3.6. Prepare Local Directories

To make mounting data/code easier, it is recommended to create the following directories locally (example paths):

```
# Create local directories (customize paths as needed)
mkdir -p /local/competition/input # Store input data (e.g., JSON, images, Excel)
mkdir -p /local/competition/output # Store model output results
mkdir -p /local/competition/code # Store participant code files (e.g., .py, .sh)
# Grant read/write permissions to directories (avoid permission issues inside the container)
chmod 777 -R /local/competition
```

4.4. Core Usage Scenarios

4.4.1. Scenario 1: Run PyTorch GPU-accelerated scripts directly (competition inference/training)

Suitable for running PyTorch GPU code in batch, non-interactive mode:

```
# Example: run /workspace/pytorch_train.py (enable all GPUs)
docker run --rm \\
--gpus all \\
-v /local/competition/input:/input \\
-v /local/competition/output:/output \\
-v /local/competition/code:/workspace \\
competition-base:pytorch2.5.1-cuda12.1-cudnn9 \\
python /workspace/pytorch_train.py
```

Run on a single GPU (to avoid occupying all resources):

```
docker run --rm \\
--gpus "device=0" # Use only GPU 0 (for multi-GPU you can specify dev
```

ice=0,1)

```
-v /local/competition/input:/input \\
-v /local/competition/output:/output \\
-v /local/competition/code:/workspace \\
competition-base:pytorch2.5.1-cuda12.1-cudnn9 \\
python /workspace/pytorch_train.py
```

4.4.2. Scenario 2: Run a custom shell script (batch PyTorch GPU tasks)

If you need to run multi-step GPU tasks (e.g., data preprocessing → PyTorch training → exporting results), you can write a shell script:

Create /local/competition/code/run_pytorch.sh locally. Example content:

```
#!/bin/bash
echo "Starting PyTorch GPU tasks..."
# Step 1: Data preprocessing (CPU)
python /workspace/preprocess.py --input /input/raw_data.json --output /input/processed_data.json
# Step 2: PyTorch GPU model training
python /workspace/pytorch_train.py --input /input/processed_data.json --output /output/model.pth
# Step 3: PyTorch GPU model inference
python /workspace/pytorch_infer.py --model /output/model.pth --input /input/test_data.json --output /output/pred_result.json
# Step 4: Validate output format
python /workspace/validate_format.py --file /output/pred_result.json
echo "PyTorch GPU tasks completed!"
```

Grant execute permission to the script:

```
chmod +x /local/competition/code/run_pytorch.sh
```

Run the script inside the container (with GPU enabled):

```
docker run --rm \\
--gpus all \\
-v /local/competition/input:/input \\
-v /local/competition/output:/output \\
```

```
-v /local/competition/code:/workspace \\
competition-base:pytorch2.5.1-cuda12.1-cudnn9 \\
/workspace/run_pytorch.sh
```

4.5. Organizer Notes

4.5.1. GPU Resource Limits

When running containers, you can limit GPU memory/compute to ensure fair evaluation:

```
docker run --rm \\
--gpus "device=0" \\
--memory=32G \\
--cpus=8 \\
--env NVIDIA_VISIBLE_DEVICES=0 \\
--env NVIDIA_MEMORY_LIMIT=16GB \\ # Limit GPU memory to 16GB
-v /local/competition/input:/input \\
-v /local/competition/output:/output \\
competition-base:pytorch2.5.1-cuda12.1-cudnn9 \\
python /workspace/pytorch_train.py
```

4.5.2. Log Collection

If you need to collect PyTorch GPU runtime logs, you can redirect the output to a file:

```
docker run --rm \\
--gpus all \\
-v /local/competition/input:/input \\
-v /local/competition/output:/output \\
competition-base:pytorch2.5.1-cuda12.1-cudnn9 \\
python /workspace/pytorch_train.py > /local/competition/output/run_pytorch.log 2>&1
```

4.5.3. CPU-Compatible Run

If you do not have a GPU, remove the --gpus option. PyTorch will automatically fall back to CPU mode (make sure your code checks for GPU availability).

4.6. Participant Notes

- Control GPU memory usage. It is recommended to limit memory fraction via torch.cuda.set_per_process_memory_fraction(), or adjust batch_size to stay within ~70% of GPU VRAM.
- Use absolute container paths for all inputs/outputs (/input, /output, /workspace). Do not use local host paths in your code.
- Use the pre-installed PyTorch environment in the image. Do not install PyTorch manually in your code to avoid version conflicts.
- After execution, make sure result files (e.g., predictions, model files) are written to /output; otherwise the evaluation system cannot read them.

4.7. Disclaimer

This document is written as a reference for adapting the competition GPU base image to a PyTorch environment. Please adjust as needed for your actual scene

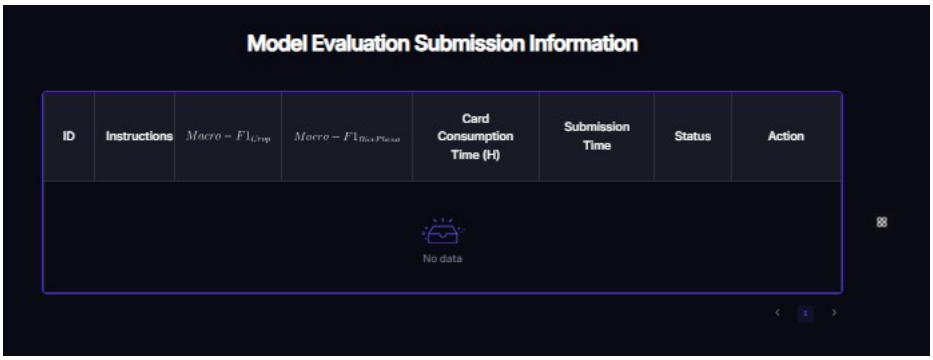
5. Other Notes on the Validation Environment

5.1. The validation environment for Track 2 taking too long to run

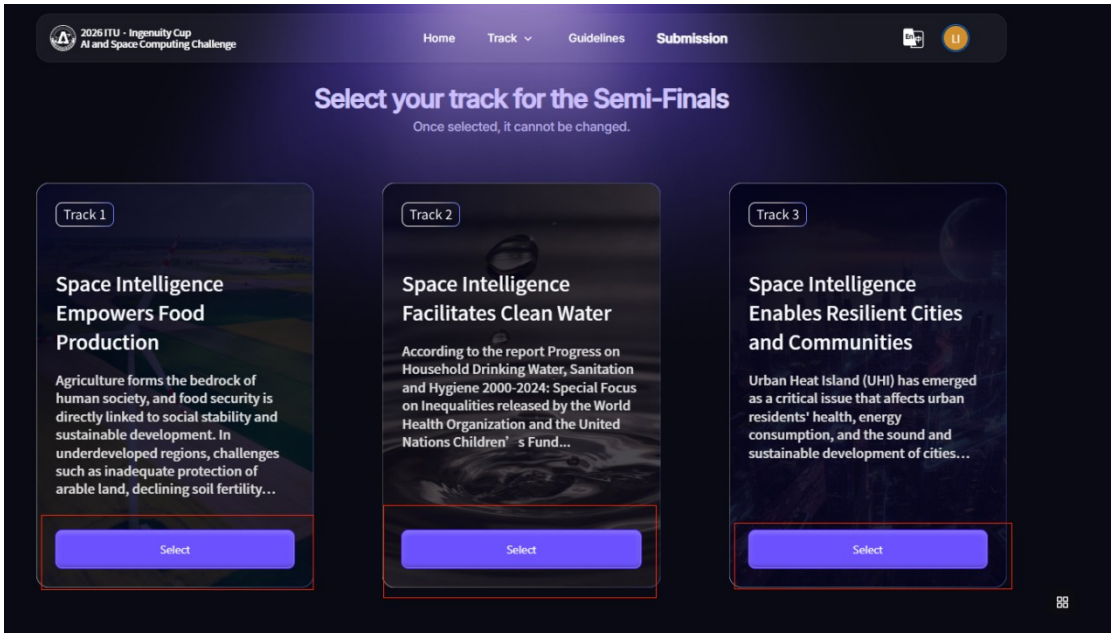
Due to the large size of the image file, the model submission phase for Track 2 takes a longer time. This only affects the submission process and does not impact inference performance. The total time from submission to result generation is expected to be approximately 1 hour.

5.2. Unable to view inference results in the validation system

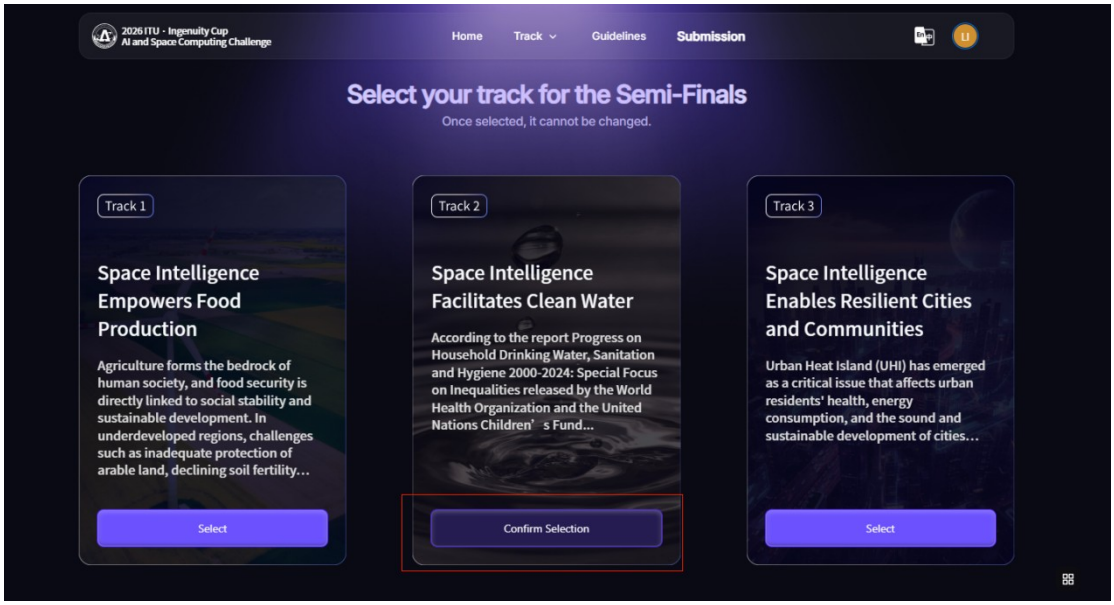
Once the validation environment finishes the submission, inference, and evaluation process, you can check the results on the competition website's submission page.。



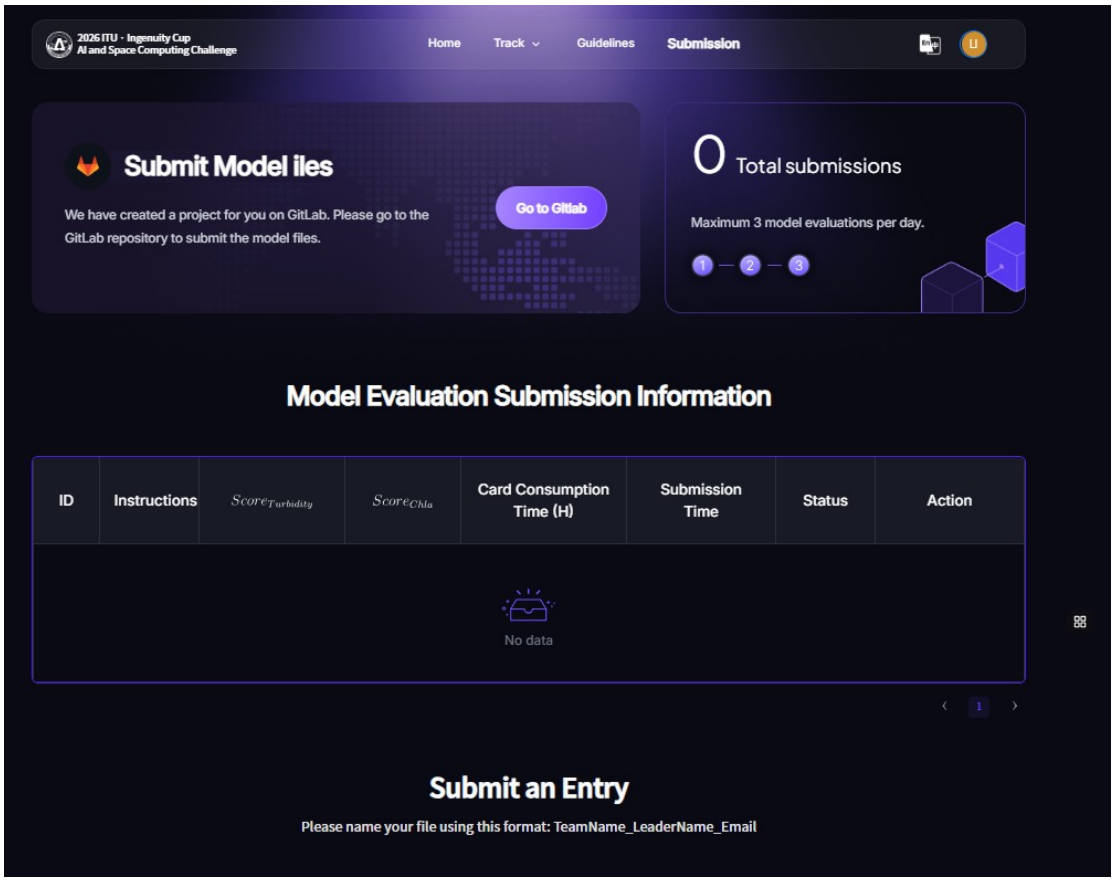
5.3. How to Select a Competition Track?



Select one of the tracks and click Select.



After clicking, you will be prompted to Confirm Selection.



Once confirmed, you will be directed to the submission page.